

Arquitectura del Sistema Lubix

Jose Armenta Farias

Johan Nicolás Acuña Duarte

Juan David Molina Sánchez

Daniel Matias Monroy Huertas

Centro de Electricidad, Electrónica y Telecomunicaciones (CEET)

Servicio Nacional de Aprendizaje (SENA)

Análisis y Desarrollo de Software (ADSO)

31 de agosto de 2026

Arquitectura del Sistema Lubix

1. Tipo de Arquitectura

El sistema Lubix ha sido diseñado e implementado bajo un esquema modular y desacoplado que garantiza el mantenimiento, la escalabilidad y la robustez técnica. A continuación, se describen las características fundamentales de la arquitectura seleccionada:

- **Arquitectura en Capas (Layered Architecture):** El sistema se organiza en capas concéntricas con responsabilidades delimitadas (Presentación, Controladores, Lógica de Negocio, Acceso a Datos y Base de Datos), evitando la interdependencia directa entre componentes heterogéneos.
- **Basada en API REST:** La comunicación entre el cliente y el servidor se realiza mediante el estándar REST over HTTP/HTTPS, utilizando verbos HTTP explícitos (GET, POST, PUT, DELETE) y transferencia de datos estructurados en formato JSON.
- **Preparada para Microservicios:** El diseño modular interno facilita una transición paulatina desde una estructura monolítica limpia hacia un enfoque distribuido basado en microservicios a medida que la demanda operativa aumente.

2. Estructura General del Sistema

La arquitectura del sistema Lubix se compone de cinco capas bien definidas. Cada capa cumple un rol específico dentro del ciclo de vida del procesamiento de datos.

2.1. Capa de Presentación (Cliente / Frontend)

Esta capa constituye la interfaz con la que interactúa el usuario final. Actualmente abarca la aplicación web desarrollada e integra la base técnica para la futura aplicación móvil. Consume

los servicios expuestos por la API mediante peticiones HTTP asíncronas y se encarga de renderizar los datos de manera amigable e intuitiva.

2.2. Capa de Controladores (Routes)

Representa la puerta de entrada formal a la aplicación. Define y expone los puntos de enlace (endpoints) como /login, /usuarios y /agendamientos. En esta capa se utiliza la librería Pydantic para la validación estricta del esquema, tipos de datos y restricciones de entrada antes de ser transferidos a la capa de negocio.

2.3. Capa de Lógica de Negocio (Services)

Corresponde al núcleo conceptual del sistema donde se ejecutan las reglas operativas y algoritmos. La capa de servicios evalúa la validez de las operaciones (tales como la disponibilidad de citas o los permisos de autenticación) y procesa los datos transformándolos previo a su almacenamiento o envío.

2.4. Capa de Acceso a Datos (Models / Repository)

Gestiona de forma abstracta la interacción con el sistema de almacenamiento. Se implementan modelos de objetos mapeados mediante SQLAlchemy (ORM) para ejecutar operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sin escribir sentencias SQL manuales complejas, manteniendo el aislamiento del motor subyacente.

2.5. Base de Datos

El sistema utiliza PostgreSQL como motor relacional primario de almacenamiento de alta fiabilidad. Se encarga de mantener la integridad referencial y la persistencia de usuarios, parámetros del sistema y agendamientos operativos correspondientes al proyecto I.S.A.

Tabla 1

Resumen de las Capas de la Arquitectura del Sistema Lubix

Capa	Tecnología / Herramienta	Responsabilidad Principal
Presentación	Aplicación Web / Móvil	Interacción con el usuario y consumo de API REST.
Controladores	FastAPI / Routes / Pydantic	Gestión de endpoints y validación de datos de entrada.
Lógica de Negocio	Python (Services)	Procesamiento de reglas de negocio y orquestación.
Acceso a Datos	SQLAlchemy (ORM)	Mapeo objeto-relacional y operaciones CRUD.
Base de Datos	PostgreSQL	Almacenamiento persistente e integridad de los datos.

Nota. Elaboración propia basada en la especificación técnica de Lubix.

3. Flujo del Sistema

El ciclo de ejecución de una petición dentro del sistema Lubix sigue una secuencia ordenada y determinista, asegurando la trazabilidad y la integridad de la información en todo momento:

1. **Solicitud Inicial:** El cliente genera una petición desde la interfaz de usuario (web o móvil) mediante un protocolo HTTP/HTTPS.
2. **Recepción y Filtrado:** La petición es interceptada por los controladores (Routes) para verificar la ruta solicitada y autenticar al cliente.
3. **Validación de Esquema:** La capa de esquemas (Pydantic) analiza la estructura del cuerpo (body) o parámetros, asegurando que los tipos de datos sean válidos.
4. **Ejecución del Negocio:** La información validada se transfiere a la capa de servicios (Services), aplicando las reglas de negocio específicas.

5. **Persistencia / Consulta:** Los servicios invocan los modelos ORM (SQLAlchemy) para interactuar directamente con la base de datos PostgreSQL.

6. **Respuesta Final:** Los datos obtenidos o el resultado de la operación son formateados e independizados para ser retornados en formato JSON al cliente.